Experiment 4: Docker Essentials

Topics Covered:
- Dockerfile
- .dockerignore
- Tagging
- Publishing

Part 1: Containerizing Applications with Dockerfile

Step 1: Create a Simple Application

Python Flask App:

```
mkdir my-flask-app
cd my-flask-app
```

`app.py`

```python
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

`requirements.txt`

```
Flask==2.3.3
```

Step 2: Create Dockerfile

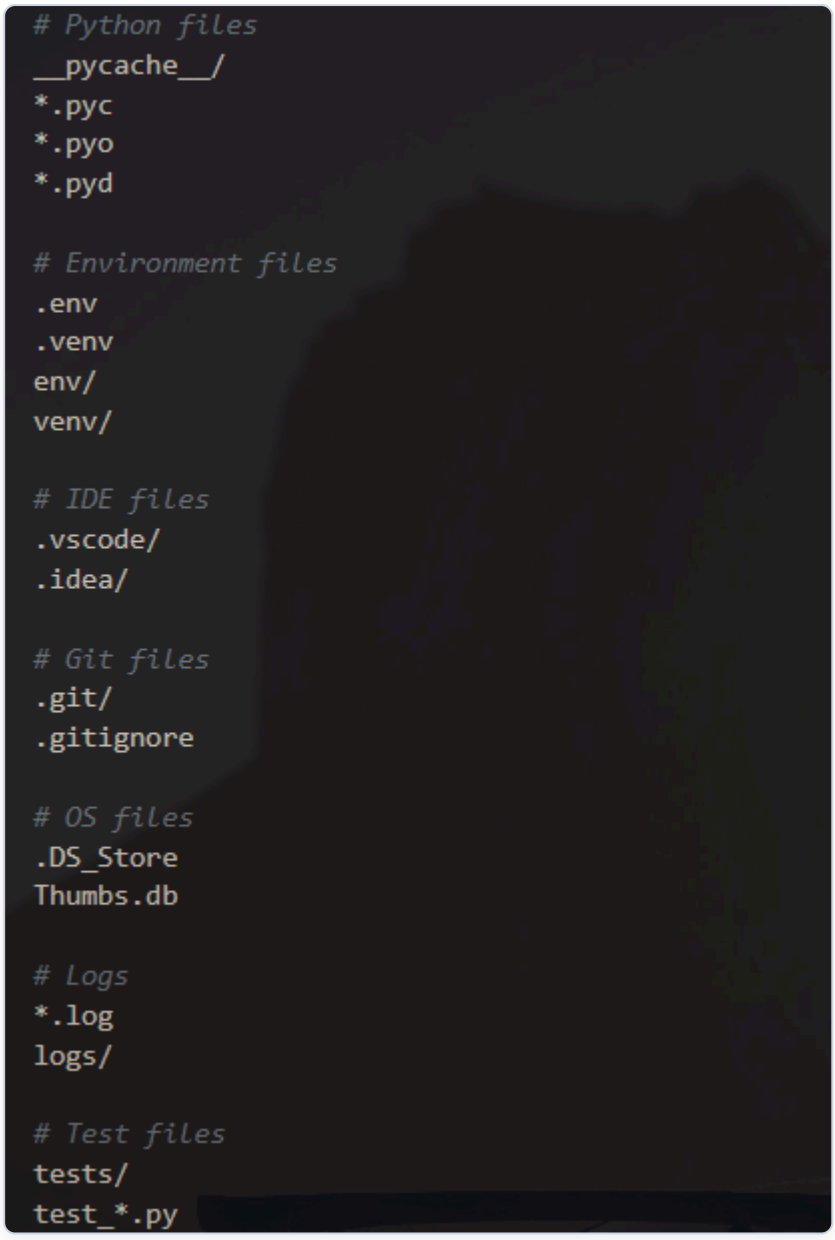
`Dockerfile`

```dockerfile
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

Part 2: Using .dockerignore

Step 1: Create .dockerignore File

`.dockerignore`

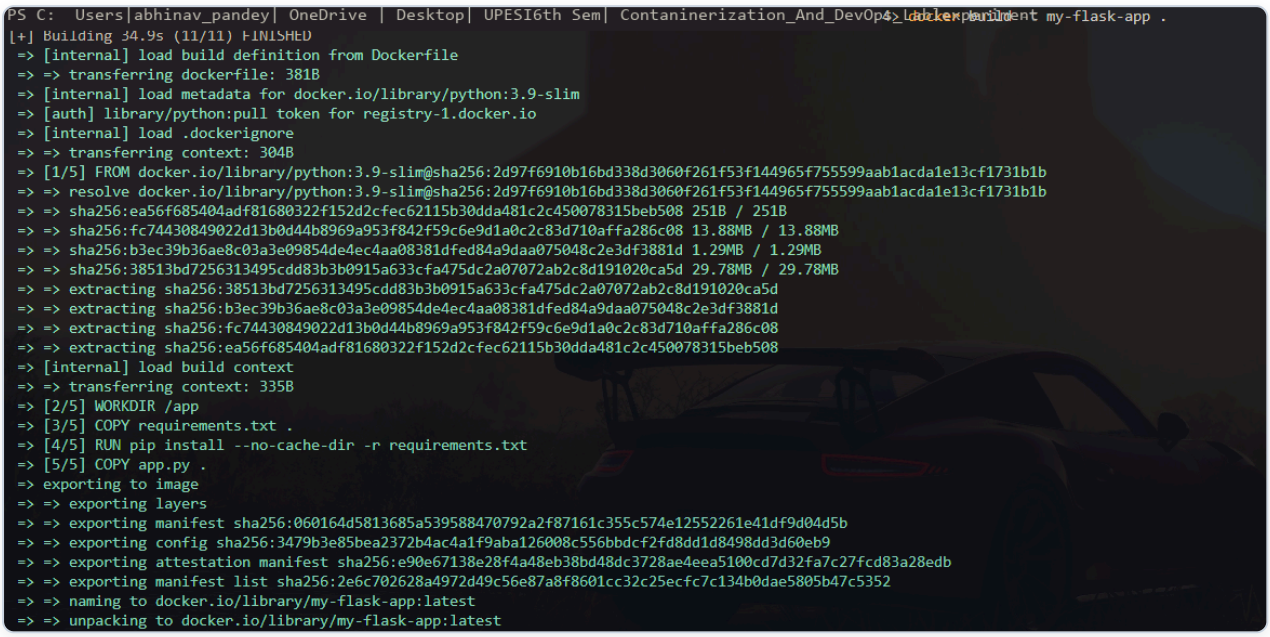


Step 2: Why .dockerignore is Important

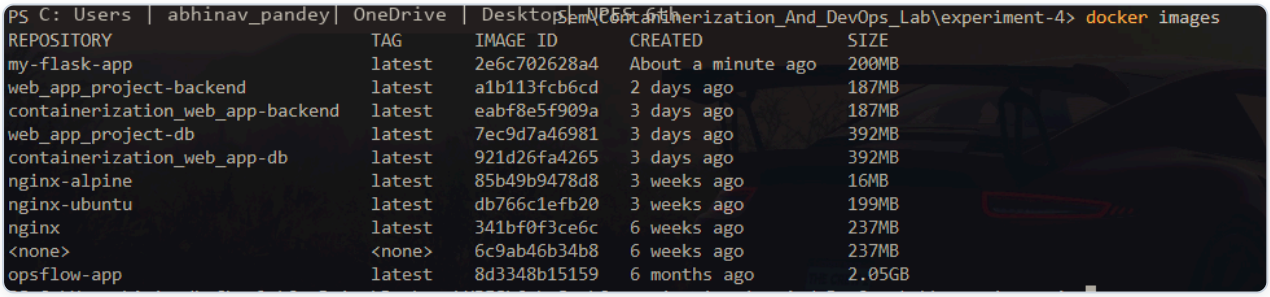
- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

Part 3: Building Docker Images

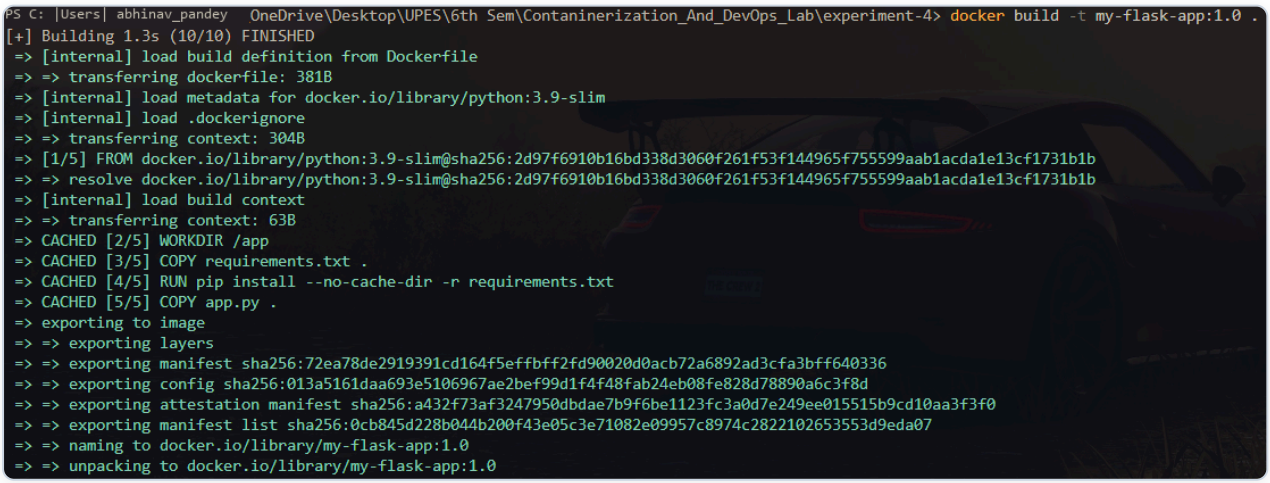
Step 1: Basic Build Command



Step 2: Tagging Images



Step 3: View Image Details



Part 4: Running Containers

Step 1: Run Container

```
PS C:\Users | abhinav_pandey  OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiment-4> # List all images
>> docker images
>>
>> # Show image history
>> docker history my-flask-app
>>
>> # Inspect image details
>> docker inspect my-flask-app
>>
REPOSITORY                          TAG       IMAGE ID       CREATED         SIZE
my-flask-app                        1.0       3179d3c3cc8f   4 minutes ago   200MB
my-flask-app                        latest    3179d3c3cc8f   4 minutes ago   200MB
my-flask-app                        v1.0      3179d3c3cc8f   4 minutes ago   200MB
username/my-flask-app               1.0       971699264f9c   4 minutes ago   200MB
web_app_project-backend             latest    a1b113fcb6cd   2 days ago      187MB
containerization_web_app-backend    latest    eabf8e5f909a   3 days ago      187MB
web_app_project-db                  latest    7ec9d7a46981   3 days ago      392MB
containerization_web_app-db         latest    921d26fa4265   3 days ago      392MB
nginx-alpine                        latest    85b49b9478d8   3 weeks ago     16MB
nginx-ubuntu                        latest    db766c1efb20   3 weeks ago     199MB
nginx                               latest    341bf0f3ce6c   6 weeks ago     237MB
<none>                              <none>    6c9ab46b34b8   6 weeks ago     237MB
opsflow-app                         latest    8d3348b15159   6 months ago    2.05GB
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
3179d3c3cc8f   4 minutes ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      4 minutes ago   EXPOSE &{[{{17 0} {17 0}}] 0xc001054280}        0B        buildkit.dockerfile.v0
<missing>      4 minutes ago   COPY app.py . # buildkit                        12.3kB    buildkit.dockerfile.v0
<missing>      4 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r…   13.4MB    buildkit.dockerfile.v0
<missing>      4 minutes ago   COPY requirements.txt . # buildkit              12.3kB    buildkit.dockerfile.v0
<missing>      4 minutes ago   WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      4 months ago    CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 months ago    RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      4 months ago    RUN /bin/sh -c set -eux;  savedAptMark="$(a…    45.7MB    buildkit.dockerfile.v0
<missing>      4 months ago    ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8…   0B        buildkit.dockerfile.v0
<missing>      4 months ago    ENV PYTHON_VERSION=3.9.25                       0B        buildkit.dockerfile.v0
<missing>      4 months ago    ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3…   0B        buildkit.dockerfile.v0
<missing>      4 months ago    RUN /bin/sh -c set -eux;  apt-get update;  a…   4.94MB    buildkit.dockerfile.v0
<missing>      4 months ago    ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 months ago    ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      5 months ago    # debian.sh --arch 'amd64' out/ 'trixie' '@1…   87.4MB    debuerreotype 0.16
[
    {
        "Id": "sha256:3179d3c3cc8f8000d22418c5c6c7241f6f8dc7f4dc8ea497f0dda002a4c21724",
        "RepoTags": [
            "my-flask-app:1.0",
            "my-flask-app:latest",
            "my-flask-app:v1.0"
        ],
        "RepoDigests": [
            "my-flask-app@sha256:3179d3c3cc8f8000d22418c5c6c7241f6f8dc7f4dc8ea497f0dda002a4c21724"
        ],
        "Parent": "",
        "Comment": "buildkit.dockerfile.v0",
```

Step 2: Manage Containers

Part 5: Multi-stage Builds

Step 1: Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

Step 2: Simple Multi-stage Dockerfile

`Dockerfile.multistage`

```dockerfile
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

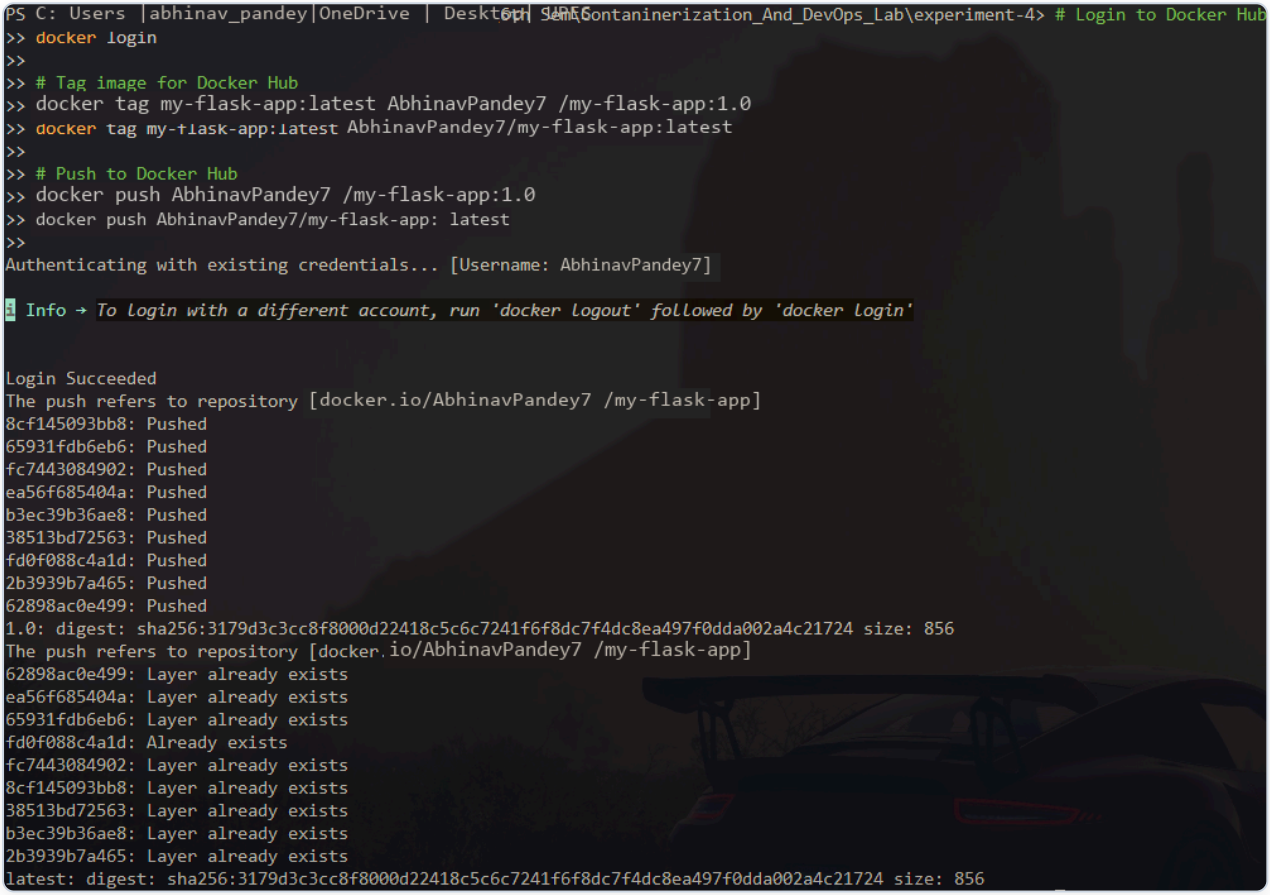
Step 3: Build and Compare

```
PS C:\Users\abhinav_pandey\OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiment
>> docker stop flask-container
>>
>> # Start stopped container
>> docker start flask-container
>>
>> # Remove container
>> docker rm flask-container
>>
>> # Remove container forcefully
>> docker rm -f flask-container
>>
flask-container
flask-container
Error response from daemon: cannot remove container "flask-container": container is running: stop the container before removing or force remove
flask-container
PS C:\Users\abhinav_pandey\OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiment> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/qvka4pbwiexffg6hxjgsiuryf
flask-multistage                latest    e17ac3145f61   1 second ago     219MB
username/my-flask-app           1.0       971699264f9c   21 minutes ago   200MB
my-flask-app                    1.0       3179d3c3cc8f   21 minutes ago   200MB
my-flask-app                    latest    3179d3c3cc8f   21 minutes ago   200MB
my-flask-app                    v1.0      3179d3c3cc8f   21 minutes ago   200MB
flask-regular                   latest    518e70674bd4   21 minutes ago   200MB
```

Part 6: Publishing to Docker Hub

Step 1: Prepare for Publishing

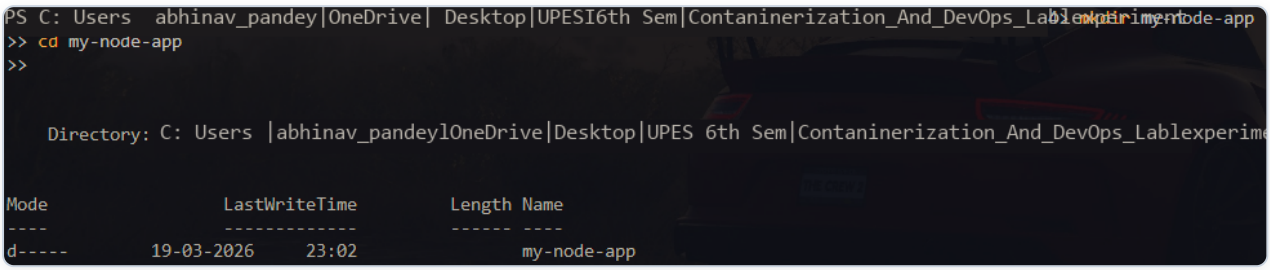


Step 2: Pull and Run from Docker Hub

```
# Pull from Docker Hub (on another machine)
docker pull username/my-flask-app:latest

# Run the pulled image
docker run -d -p 5000:5000 username/my-flask-app:latest
```

Part 7: Node.js Example (Quick Version)

Step 1: Node.js Application



`app.js`

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
    res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
    res.json({ status: 'healthy' });
});

app.listen(port, () => {
    console.log(`Server running on port ${port}`);
});
```

`package.json`

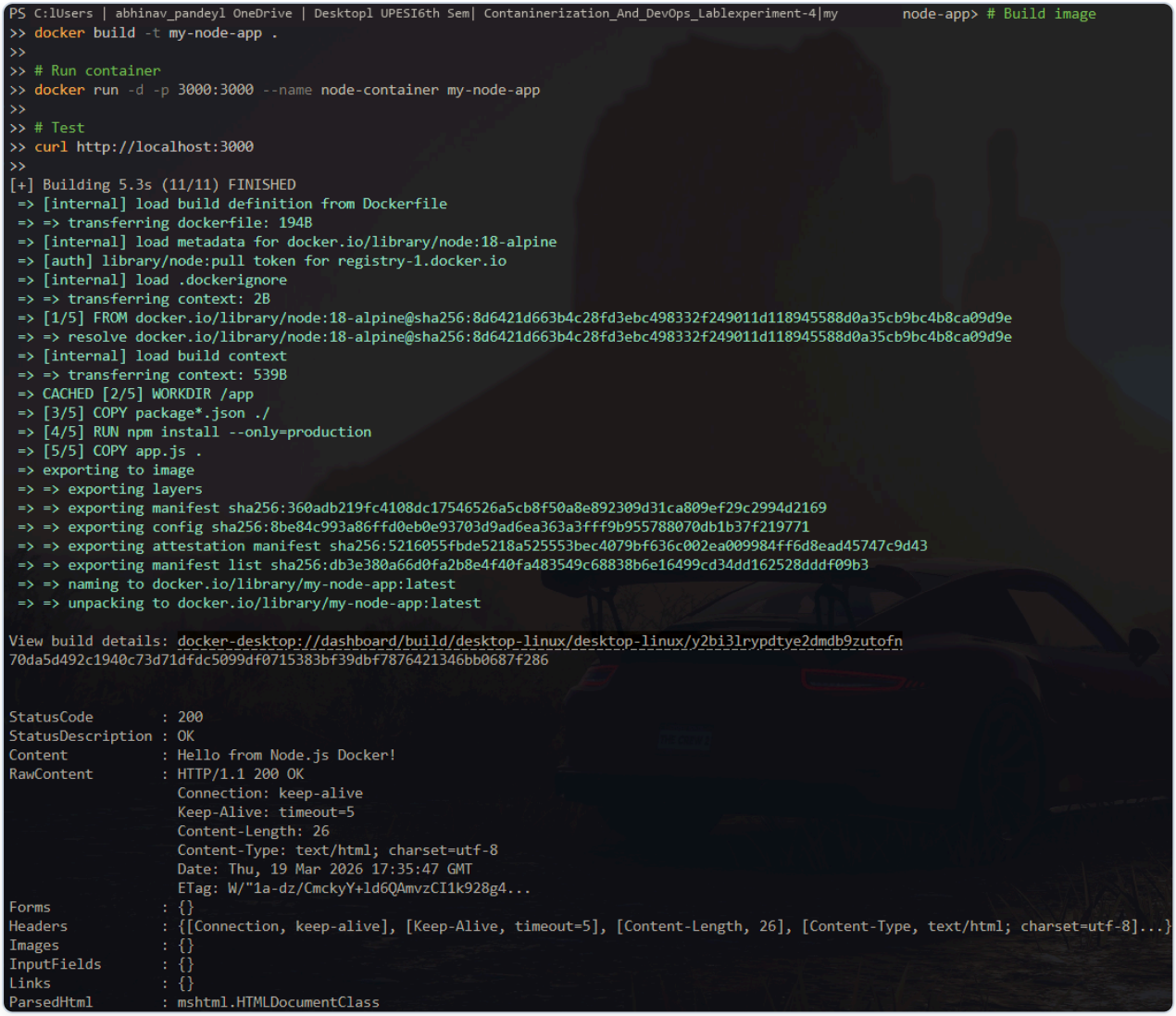
```
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

Step 2: Node.js Dockerfile

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

Step 3: Build and Run

```
PS C:\Users\abhinav_pandey\OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiments> cd my-node-app
>> cd my-node-app
>>


    Directory: C:\Users\abhinav_pandey\OneDrive\Desktop\UPES 6th Sem\Containerization_And_DevOps_Lab\experiments\my-node-app


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        19-03-2026     23:02                my-node-app
```



Part 8: Quick Practice Exercises

Exercise 1: Tagging Practice

```
# Create an image with three tags:
# 1. myapp:latest
# 2. myapp:v2.0
# 3. yourusername/myapp:production

# Solution:
docker build -t myapp:latest -t myapp:v2.0 -t username/myapp:production .
```

Exercise 2: Multi-stage for Node.js

```
# Create a multi-stage Dockerfile for Node.js that:
# 1. Uses builder stage for npm install
# 2. Creates final image with only production dependencies
# 3. Uses non-root user

# Hint:
# STAGE 1: FROM node:18-alpine AS builder
# STAGE 2: FROM node:18-alpine
# COPY --from=builder /app/node_modules ./node_modules
```

Exercise 3: Clean Build

```
# Build without cache and with .dockerignore
docker build --no-cache -t clean-app .

# Compare with cached build
time docker build -t cached-app .
```

Essential Docker Commands Cheatsheet

Command	Purpose	Example
`docker build`	Build image	`docker build -t myapp .`
`docker run`	Run container	`docker run -p 3000:3000 myapp`
`docker ps`	List containers	`docker ps -a`
`docker images`	List images	`docker images`
`docker tag`	Tag image	`docker tag myapp:latest myapp:v1`
`docker login`	Login to Dockerhub	`echo "token" | docker login -u username --password-stdin`
`docker push`	Push to registry	`docker push username/myapp`
`docker pull`	Pull from registry	`docker pull username/myapp`
`docker rm`	Remove container	`docker rm container-name`
`docker rmi`	Remove image	`docker rmi image-name`
`docker logs`	View logs	`docker logs container-name`
`docker exec`	Execute command	`docker exec -it container-name bash`

Common Workflow Summary

Development Workflow:

```
# 1. Create Dockerfile and .dockerignore
# 2. Build image
docker build -t myapp .

# 3. Test locally
docker run -p 8080:8080 myapp

# 4. Tag for production
docker tag myapp:latest myapp:v1.0

# 5. Push to registry
docker push myapp:v1.0
```

Production Workflow:

```
# 1. Pull from registry
docker pull myapp:v1.0

# 2. Run in production
docker run -d -p 80:8080 --name prod-app myapp:v1.0

# 3. Monitor
docker logs -f prod-app
```

Key Takeaways

1. **Dockerfile** defines how to build your image.
2. **.dockerignore** excludes unnecessary files (important for security and performance).
3. **Tagging** helps version and organize images.
4. **Multi-stage builds** create smaller, more secure production images.
5. **Docker Hub** allows sharing and distributing images.
6. Always test images locally before publishing.

Cleanup

```
# Remove all stopped containers
docker container prune

# Remove unused images
docker image prune

# Remove everything unused
docker system prune -a
```